

TypeScript Level-3 – Complete Documentation (Java Developer Perspective)

This document completes **TypeScript preparation before NestJS**. It focuses on **design-time typing**, **API contracts**, and **immutability**—areas critical for real-world backend systems.

1. `type` vs `interface` (INTERVIEW FAVORITE)

Interface

```
interface User {  
  id: number;  
  name: string;  
}
```

Type

```
type User = {  
  id: number;  
  name: string;  
};
```

Key Differences

Aspect	interface	type
Declaration merging	✓ Yes	✗ No
Extends / inheritance	<code>extends</code>	<code>&</code> (intersection)
Use in classes	✓ Preferred	⚠ Allowed
Unions / primitives	✗ No	✓ Yes

When to use (Rule of Thumb)

- **interface** → Public contracts (DTOs, Entities)
- **type** → Complex unions, utility composition

2. Extending vs Intersections

Interface extends

```
interface Base {  
  id: number;  
}  
  
interface User extends Base {  
  name: string;  
}
```

Type intersection

```
type Base = { id: number };  
type User = Base & { name: string };
```

3. Utility Types (VERY IMPORTANT)

3.1 `Partial<T>`

Makes all properties optional.

```
interface User {  
  id: number;  
  name: string;  
  age: number;  
}  
  
const updateUser: Partial<User> = {  
  name: 'Rohit'  
};
```

NestJS use: Update DTOs

3.2 `Required<T>`

Makes all properties required.

```
type RequiredUser = Required<User>;
```

3.3 Pick<T, K>

Select specific fields.

```
type UserPreview = Pick<User, 'id' | 'name'>;
```

Use: API response shaping

3.4 Omit<T, K>

Remove fields.

```
type UserWithoutAge = Omit<User, 'age'>;
```

Use: Hide sensitive data (passwords)

3.5 Readonly<T>

Prevents mutation.

```
type ReadonlyUser = Readonly<User>;
```

4. Enums vs Literal Unions

Enum

```
enum Role {  
  ADMIN = 'ADMIN',  
  USER = 'USER'  
}
```

Literal Union (Preferred)

```
type Role = 'ADMIN' | 'USER';
```

Why literal unions are better

- No runtime JS output
- Tree-shakable
- Better type inference

NestJS best practice: Prefer literal unions

5. Const Assertions (`as const`)

```
const ROLES = ['ADMIN', 'USER'] as const;  
  
type Role = typeof ROLES[number];
```

Creates **exact literal types**.

6. Immutability Patterns

Mutable (avoid)

```
user.age = 30;
```

Immutable (preferred)

```
const updatedUser = { ...user, age: 30 };
```

Deep immutability

```
const config = {  
  env: 'prod'  
} as const;
```

7. Record<K, V> (Maps)

```
type RolePermissions = Record<Role, string[]>;

const permissions: RolePermissions = {
  ADMIN: ['read', 'write'],
  USER: ['read']
};
```

Java equivalent: `Map<Role, List<String>>`

8. Index Signatures

```
interface ErrorMap {
  [key: string]: string;
}
```

9. Discriminated Unions (POWERFUL)

```
type ApiResponse =
  | { status: 'success'; data: User }
  | { status: 'error'; message: string };
```

Compiler automatically narrows type.

10. Real-World NestJS-Style Example

```
type CreateUserDto = {
  name: string;
  age: number;
};

type UpdateUserDto = Partial<CreateUserDto>;

type UserResponse = Omit<CreateUserDto, 'age'> & {
  id: number;
};
```

Uses: - Partial - Omit - Composition

11. Java Comparison Summary

Java	TypeScript
POJO	Interface / Type
Builder pattern	Partial + spread
Enum	Literal unions
Immutability via final	Readonly / spread

12. Interview One-Liners

- "I prefer interfaces for public contracts and types for complex compositions."
 - "Utility types like Partial and Omit help create clean update DTOs."
 - "Literal unions are safer than enums in TypeScript backend systems."
-

13. Readiness Checklist (FINAL)

After this document, you should be confident with: - type vs interface - Utility types - Enums vs unions - Immutability - DTO design patterns

 **TypeScript is now COMPLETE**

Next Step

 Start **NestJS from scratch**  Project-based learning with controllers, services, modules, DI